

Fase 16 — Consultas Distribuidas

Materia

BDD

Continuación directa de la Fase 15. Los seis nodos del laboratorio tienen sus snapshots `fase15-completa` tomados: `bdd-nodo01` - `03` conforman el clúster Galera con `lab_bdd` completo replicado, `bdd-nodo04` y `bdd-nodo05` son los shards con la fragmentación horizontal y vertical implementada, y `bdd-nodo06` actúa como coordinador Spider con las tablas distribuidas, las VIEWS de reconstrucción, la vista `reporte_pedidos_detallado` y el procedimiento `consulta_regional()` creados en la Fase 15. En esta fase **no se redistribuyen datos ni se modifica la arquitectura de fragmentación**: el objetivo es formalizar el **algoritmo de procesamiento de consultas distribuidas** en sus cuatro fases clásicas, implementar manualmente la técnica de **semijoin distribuida**, medir el coste real de transferencia de datos con variables de estado de MariaDB, y provisionar `bdd-nodo07` (`bdd-cliente`)—la estación cliente externa que completa el mapa de nodos del laboratorio—como prueba final de transparencia de distribución. Al finalizar la fase, cualquier consulta de negocio sobre `lab_bdd` puede emitirse desde `bdd-cliente` hacia `bdd-nodo06` sin que la aplicación cliente perciba que los datos residen en cuatro nodos físicamente separados.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Aplicar el **algoritmo de procesamiento de consultas distribuidas** de cuatro fases —descomposición, localización, optimización global y ejecución— sobre la consulta híbrida canónica del laboratorio, identificando en qué fase ocurre cada transformación y qué subconsultas SQL representan cada etapa.
2. Simular manualmente la **fase de localización**: sustituir cada relación global por su expresión de fragmentos y aplicar predicados de simplificación para eliminar fragmentos vacíos antes de ejecutar ninguna subconsulta remota.
3. Implementar la técnica de **semijoin distribuida** (algoritmo de Bernstein-Chiu) paso a paso, comparando el volumen de datos transferidos entre el coordinador y los shards con y sin reducción, y calculando el beneficio concreto para el esquema del laboratorio.
4. Calcular el **coste estimado de transferencia** para distintos planes de ejecución usando `information_schema.TABLES` (`TABLE_ROWS` y `AVG_ROW_LENGTH`), y contrastar la estimación teórica con las métricas de ejecución real obtenidas de las variables `Handler_read_*`.

5. Demostrar la **poda por índice en subconsultas remotas**: ejecutar localmente en cada shard la subconsulta equivalente a la que Spider le enviaría, y verificar con `EXPLAIN` que el plan local usa el índice compuesto correcto creado en la Fase 15.
 6. Comparar la **proyección temprana** frente a `SELECT *`: cuantificar la reducción en bytes cuando Spider transmite solo las columnas necesarias en lugar del ancho completo de cada fila.
 7. Crear el procedimiento `analizar_consulta()` en `bdd-nodo06` que, dado un predicado de región y un indicador de necesidad de columnas de detalle, calcula y muestra el plan de descomposición estimado con fragmentos involucrados, filas esperadas, bytes estimados y grado de localidad.
 8. Provisionar `bdd-nodo07` (`bdd-cliente`) como estación cliente ligera —sin servidor MariaDB— con la IP `192.168.56.107`, y configurarla para conectar exclusivamente a `bdd-nodo06` usando las credenciales de `app_final`.
 9. Verificar la **transparencia completa de distribución** —fragmentación, ubicación y replicación— desde `bdd-cliente` ejecutando ocho pruebas que confirman que la aplicación cliente no puede distinguir un sistema distribuido de uno centralizado ni acceder a la infraestructura interna de los shards.
 10. Cerrar la fase con el snapshot `fase16-completa` en los **siete nodos** del laboratorio, incluyendo el primer snapshot de `bdd-nodo07`.
-

B. Conceptos teóricos necesarios

1. Las cuatro fases del procesamiento de consultas distribuidas.

El procesamiento de una consulta SQL en un sistema distribuido sigue un flujo de cuatro fases secuenciales (Özsu & Valduriez, *Principles of Distributed Database Systems*, 2011):

Fase	Nombre	Qué pregunta responde	Resultado
1	Descomposición	¿Cómo se transforma el SQL en álgebra relacional optimizada?	Árbol algebraico con selecciones y proyecciones empujadas hacia las hojas
2	Localización	¿En qué fragmentos residen los datos de cada relación global?	Árbol con relaciones globales sustituidas por expresiones de fragmentos; fragmentos vacíos eliminados
3	Optimización global	¿En qué nodo se ejecuta cada operación? ¿Qué orden de join minimiza el coste de red?	Plan de ejecución distribuido con asignación de subconsultas a nodos
4	Ejecución	¿Cómo se despachan las subconsultas y se ensambla el resultado?	Envío a nodos, recepción de resultados parciales, ensamblado en el coordinador

Spider implementa automáticamente las fases 3 y 4. Esta fase simula manualmente las cuatro fases para hacer explícito el razonamiento interno del motor.

2. Álgebra relacional distribuida: equivalencias de optimización.

El optimizador usa equivalencias formales para reducir el coste sin cambiar el resultado semántico:

- *Cascada de selecciones*: $\sigma(p \wedge q)(R) = \sigma(p)(\sigma(q)(R))$ — permite aplicar predicados de fragmentación de forma independiente antes de unir fragmentos.
- *Conmutatividad selección-join*: $\sigma(p)(R \bowtie S) = \sigma(p_R)(R) \bowtie \sigma(p_S)(S)$ cuando $p = p_R \wedge p_S$ — permite empujar predicados a cada fragmento antes del join.

- **Distributividad del join sobre la unión:** $R \bowtie (S_1 \cup S_2) = (R \bowtie S_1) \cup (R \bowtie S_2)$ — permite hacer joins locales en cada shard antes de unir resultados.
- **Proyección temprana:** $\pi_A(R \bowtie S) = \pi_A(\pi_{\{A_R \cup \text{join_attrs}\}}(R) \bowtie \pi_{\{A_S \cup \text{join_attrs}\}}(S))$ — reduce el ancho de filas enviadas por la red.

3. Semijoin distribuida: definición formal y algoritmo.

El semijoin de R con S se define como:

Text

$$R \ltimes S = \{r \in R \mid \exists s \in S : r[\text{attr_join}] = s[\text{attr_join}]\}$$

En SQL: $R \ltimes S \equiv \text{SELECT } * \text{ FROM } R \text{ WHERE attr_join IN (SELECT attr_join FROM S)}$.

El algoritmo clásico de Bernstein-Chiu (1981) para dos nodos N_1 (contiene R) y N_2 (contiene S) unidos por atributo a :

Text

```
Paso 1:  $S' = \pi_a(S)$  → solo los IDs de join en  $N_2$  (pequeño)
Paso 2: Transferir  $S'$  de  $N_2$  a  $N_1$ 
Paso 3:  $R' = R \ltimes S'$  → filtrar R localmente en  $N_1$  (solo
filas coincidentes)
Paso 4: Transferir  $R'$  de  $N_1$  a  $N_2$ 
Paso 5: Resultado =  $R' \bowtie S$  → join completo en  $N_2$  o en el coordinador
```

El beneficio del semijoin es positivo cuando:

Text

$$\text{size}(S) > \text{size}(S') + \text{size}(R')$$

Es decir, cuando el join tiene alta selectividad (pocos registros de R coinciden con S) o cuando S contiene muchas columnas que N_1 no necesita. *Nota:* Spider de MariaDB no aplica semijoin automáticamente; en esta fase se implementa manualmente para demostrar el concepto.

4. Estimación del coste de comunicación.

El modelo de coste de transferencia simplificado (sin latencia) es:

Text

$$C_{\text{transfer}}(R) = |R| \times \text{AVG_ROW_LENGTH}(R) \quad [\text{bytes}]$$

Ambos valores están disponibles en `information_schema.TABLES` tras ejecutar `ANALYZE TABLE`. En un entorno WAN, el coste de latencia domina; en la red virtual Host-Only del laboratorio, el ancho de banda es virtualmente ilimitado, por lo que se usa el volumen en bytes como proxy de coste comparativo.

5. Poda por índice en subconsultas remotas.

Cuando Spider envía una subconsulta a un shard, el motor InnoDB del shard la ejecuta con su propio optimizador y sus propios índices. Los índices compuestos creados en la Fase 15 (`idx_region_cliente`, `idx_region_id`) permiten al shard devolver al coordinador exactamente las filas necesarias, sin scans completos del fragmento:

- Sin índice: shard hace full scan del fragmento, devuelve todas las filas, el coordinador filtra.
- Con `idx_region_id (region, id)`: shard hace range scan por región, devuelve solo las filas del predicado.

En tablas de volumen pequeño (10 filas en nuestro laboratorio), el optimizador InnoDB puede preferir el full scan; el principio se vuelve crítico con fragmentos de miles o millones de filas.

6. Proyección temprana y reducción de ancho de columna.

Spider empuja la lista de columnas del `SELECT` al shard. Si la consulta solo necesita `id`, `nombre`, `region` de `clientes` (~50 bytes/fila) en lugar de `SELECT *` (~120 bytes/fila), el shard devuelve al coordinador aproximadamente el 42% del volumen, independientemente del número de filas. La regla práctica: usar siempre listas de columnas explícitas en las VIEWS y procedimientos del coordinador.

7. Variables de estado de MariaDB para análisis de consultas.

Las variables `Handler_*` son contadores de sesión que se acumulan y se resetean con `FLUSH STATUS`:

Variable	Descripción
<code>Handler_read_rnd_next</code>	Filas leídas en scans secuenciales (full table scan)
<code>Handler_read_key</code>	Lookups por índice (clave primaria o índice secundario)
<code>Handler_read_next</code>	Lecturas secuenciales tras un lookup por rango en índice
<code>Bytes_sent</code>	Bytes enviados al cliente en esta sesión

Patrón de uso para comparar dos planes alternativos:

 Bash

```
FLUSH STATUS;
[CONSULTA A]
SHOW STATUS WHERE Variable_name IN
('Handler_read_rnd_next','Handler_read_key','Bytes_sent');
FLUSH STATUS;
[CONSULTA B]
SHOW STATUS WHERE Variable_name IN
('Handler_read_rnd_next','Handler_read_key','Bytes_sent');
```

8. El cliente distribuido y la transparencia completa.

`bdd-cliente` (`bdd-nodo07` , `192.168.56.107`) representa cualquier aplicación —backend web, script de análisis, herramienta BI— que consume datos del sistema sin conocimiento de la distribución interna. Solo sabe la IP de `bdd-nodo06` . Solo tiene instalado el cliente MariaDB. Al finalizar esta fase, el laboratorio implementa los tres primeros tipos de transparencia del catálogo de la Fase 0:

Tipo	Mecanismo	Estado
Fragmentación	El cliente consulta <code>clientes</code> , <code>productos</code> , etc. sin saber que los datos están divididos	Demostrado en Fases 14–15; verificado externamente en esta fase
Ubicación	El cliente solo conoce la IP de nodo06	Verificado externamente en esta fase
Replicación	nodo01–03 replican <code>lab_bdd</code> sin que el cliente lo perciba	Implementado en Fases 10–11
Concurrencia	MVCC de InnoDB; pruebas de aislamiento	Se aborda en Fase 17

C. Prerrequisitos

Estado requerido de las máquinas virtuales

Nodo	Snapshot	Estado funcional esperado
<code>bdd-nodo01</code>	<code>fase15-completa</code>	MAESTRO Galera; <code>lab_bdd</code> completo (20/10/20/36 filas); <code>binary</code> <code>log</code> activo
<code>bdd-nodo02</code>	<code>fase15-completa</code>	Miembro Galera; réplica completa; <code>read_only =</code> <code>OFF</code>
<code>bdd-nodo03</code>	<code>fase15-completa</code>	Miembro Galera; réplica completa
<code>bdd-nodo04</code>	<code>fase15-completa</code>	SHARD-A: <code>clientes</code> (10, norte+este), <code>pedidos</code> (10), <code>detalle_pedido</code> <code>s</code> (~18), <code>productos</code> (10 — se elimina en esta fase), <code>v_productos_ba</code> <code>sico</code> (10); índices <code>idx_region_cli</code> <code>ente</code> , <code>idx_region_id</code> , <code>idx_categoria</code> , <code>idx_precio</code> ; <code>server_id=4</code> ; <code>bind-</code> <code>address=0.0.0.</code> <code>0</code>

bdd-nodo05	fase15- completa	SHARD-B: <pre> clientes (10, sur+oeste), pedidos (10), detalle_pedido s (~18), productos (10 — se elimina en esta fase), v_productos_de talle (10); índices idx_region_cli ente , idx_region_id ; server_id=5 ; bind- address=0.0.0. 0 </pre>
bdd-nodo06	fase15- completa	COORDINADOR Spider: tablas Spider <pre> clientes , pedidos (PARTITION), spider_detalle _nodo04/05 , v_productos_ba sico/detalle ; </pre> VIEWs <pre> detalle_pedido s , productos , reporte_pedido s_detallado ; </pre> PROCEDURE <pre> consulta_regio nal() ; USER app_final ; server_id=6 </pre>
bdd-nodo07	—	No existe aún; se crea en esta fase como bdd- cliente

`bdd-nodo01`, `bdd-nodo02` y `bdd-nodo03` permanecen **apagados** durante los pasos de consultas distribuidas de esta fase. Solo `bdd-nodo04`, `bdd-nodo05` y `bdd-nodo06` necesitan estar activos. Se toman sus snapshots al final de la fase sin necesidad de encenderlos.

Conocimiento técnico requerido

- Catálogo de consultas H-1 a H-8 y grados de localidad (Fase 15).
- Motor Spider, `CREATE SERVER`, tablas Spider particionadas y simples, VIEWS de reconstrucción (Fase 14).
- Índices compuestos en los shards creados en la Fase 15.
- Creación de clones enlazados de VirtualBox y reconfiguración de hostname e IP (Fases 13 y 14).

D. Procedimiento paso a paso

Paso 1 — Iniciar nodo04, nodo05 y nodo06; verificar el estado heredado de la Fase 15 (conteos de filas, índices, tablas y VIEWS del coordinador).

Paso 2 — Eliminar la tabla `productos` original de nodo04 y nodo05: fue cargada como solución temporal en la Fase 13; con `v_productos_basico` y `v_productos_detalle` implementando la fragmentación vertical definitiva, la tabla original es redundante y puede causar confusión.

Paso 3 — Ejecutar el **análisis estadístico de costes**: ejecutar `ANALYZE TABLE` en todos los fragmentos y consultar `information_schema.TABLES` para construir la tabla de costes de transferencia estimados por fragmento.

Paso 4 — Simular manualmente las **cuatro fases de descomposición** sobre la consulta canónica H-6: documentar cada fase en SQL ejecutable y verificar que la Fase 4 (ejecución manual) produce exactamente el mismo resultado que la consulta H-6 directa vía Spider.

Paso 5 — Implementar la **semijoin distribuida manual** (algoritmo de Bernstein-Chiu) para el JOIN `detalle_pedidos` $\bowtie$ `v_productos_basico`, midiendo con `Handler_*` el I/O de la estrategia naive frente a la estrategia con semijoin.

Paso 6 — Comparar **proyección temprana vs.** `SELECT *`: ejecutar la misma consulta con lista explícita de columnas y con `SELECT *`, midiendo `Bytes_sent` en cada caso.

Paso 7 — Verificar la **poda por índice** conectando directamente a nodo04 y ejecutando las subconsultas que Spider enviaría, confirmando con `EXPLAIN` que el plan local usa el índice correcto.

Paso 8 — Crear el procedimiento `analizar_consulta()` en nodo06: calcula el plan de descomposición estimado, los fragmentos accedidos, los bytes de transferencia y el grado de localidad para cualquier región y configuración de columnas.

Paso 9 — Crear `bdd-nodo07` (`bdd-cliente`) como clon enlazado de `bdd-nodo01` desde el snapshot `fase06-completa` (Ubuntu Server + SSH, sin MariaDB instalado). Asignarle la IP `192.168.56.107` y el hostname `bdd-cliente`.

Paso 10 — Instalar `mariadb-client` en `bdd-cliente`, configurar `/etc/hosts` con todas las entradas del laboratorio y verificar la conectividad hacia `bdd-nodo06`.

Paso 11 — Ejecutar desde `bdd-cliente` las **ocho pruebas de transparencia**: el usuario `app_final` conectado únicamente a nodo06 ejecuta todas las operaciones de negocio sin conocimiento de la distribución interna, y se verifica que no puede acceder a la infraestructura de los shards.

Paso 12 — Apagar todos los nodos activos y tomar el snapshot `fase16-completa` en los **siete nodos** del laboratorio.

E. Comandos completos

Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan en el prompt del motor.

E.1 Iniciar nodo04, nodo05 y nodo06; verificar el estado de la Fase 15 (host + VM)

 PowerShell

```
VBoxManage startvm "bdd-nodo04" --type headless
VBoxManage startvm "bdd-nodo05" --type headless
VBoxManage startvm "bdd-nodo06" --type headless
Start-Sleep -Seconds 35
```

Conectarse a los tres nodos en terminales simultáneas:

 PowerShell

```
ssh bddadmin@192.168.56.104    # Terminal 1 - nodo04
ssh bddadmin@192.168.56.105    # Terminal 2 - nodo05
ssh bddadmin@192.168.56.106    # Terminal 3 - nodo06
```

Verificar el estado en **nodo04**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Tablas y conteos
SHOW TABLES;
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM detalle_pedidos
UNION ALL
SELECT 'productos',        COUNT(*)          FROM productos
UNION ALL
SELECT 'v_productos_basico', COUNT(*)
FROM v_productos_basico;
-- Índices actuales
SELECT TABLE_NAME,
       INDEX_NAME,
       GROUP_CONCAT(COLUMN_NAME ORDER BY SEQ_IN_INDEX SEPARATOR ', ')
AS columnas
FROM   information_schema.STATISTICS
WHERE  TABLE_SCHEMA = 'lab_bdd'
GROUP BY TABLE_NAME, INDEX_NAME
ORDER BY TABLE_NAME, INDEX_NAME;
EOF

sudo mariadb -e "SHOW VARIABLES LIKE 'server_id'; SHOW VARIABLES
LIKE 'bind_address';"
```

Verificar en **nodo05** (mismas consultas, salida esperada con **sur** y **oeste**):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
SHOW TABLES;
SELECT 'clientes'           AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'pedidos',           COUNT(*)           FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',   COUNT(*)           FROM detalle_pedidos
UNION ALL
SELECT 'productos',         COUNT(*)           FROM productos
UNION ALL
SELECT 'v_productos_detalle', COUNT(*)
FROM v_productos_detalle;
SELECT TABLE_NAME,
       INDEX_NAME,
       GROUP_CONCAT(COLUMN_NAME ORDER BY SEQ_IN_INDEX SEPARATOR ', ')
AS columnas
FROM   information_schema.STATISTICS
WHERE  TABLE_SCHEMA = 'lab_bdd'
GROUP BY TABLE_NAME, INDEX_NAME
ORDER BY TABLE_NAME, INDEX_NAME;
EOF
```

Verificar el coordinador en **nodo06**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Objetos existentes en el coordinador
SELECT TABLE_NAME AS objeto,
       TABLE_TYPE AS tipo,
       ENGINE
FROM   information_schema.TABLES
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER  BY TABLE_TYPE DESC, TABLE_NAME;
SELECT ROUTINE_NAME AS procedimiento, ROUTINE_TYPE
FROM   information_schema.ROUTINES
WHERE  ROUTINE_SCHEMA = 'lab_bdd';
-- Conteos globales desde el coordinador (prueba de conectividad Spider)
SELECT 'clientes (Spider PARTITION)' AS fuente, COUNT(*) AS filas
FROM clientes
UNION ALL
SELECT 'pedidos (Spider PARTITION)', COUNT(*)
FROM pedidos
UNION ALL
SELECT 'detalle_pedidos (VIEW UNION ALL)', COUNT(*)
FROM detalle_pedidos
UNION ALL
SELECT 'v_productos_basico (Spider-nodo04)', COUNT(*)
FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle (Spider-nodo05)', COUNT(*)
FROM v_productos_detalle
UNION ALL
SELECT 'productos (VIEW JOIN)', COUNT(*)
FROM productos;
-- Esperado: 20 / 20 / ~36 / 10 / 10 / 10
EOF
```

E.2 Eliminar la tabla `productos` original de los shards (VM — nodo04 y nodo05)

La tabla `productos` fue cargada en los shards en la Fase 13 como catálogo temporal. Con la fragmentación vertical definitiva implementada en la Fase 14 (`v_productos_basico` en nodo04, `v_productos_detalle` en nodo05), esa tabla ya no se necesita ni en nodo04 ni en nodo05.

En **nodo04**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Verificar que v_productos_basico tiene los mismos IDs que productos antes
de eliminar
SELECT 'productos (original)'      AS tabla,      COUNT(*) AS filas
FROM productos
UNION ALL
SELECT 'v_productos_basico (frag)', COUNT(*) FROM v_productos_basico;
-- Confirmar que no existen IDs en productos que no estén
en v_productos_basico
SELECT COUNT(*) AS ids_solo_en_productos_original
FROM   productos p
LEFT   JOIN v_productos_basico vb ON vb.id = p.id
WHERE  vb.id IS NULL;
-- Debe devolver 0
-- Eliminar la tabla temporal original del shard
DROP TABLE IF EXISTS productos;
-- Verificar que el shard tiene exactamente cuatro tablas definitivas
SHOW TABLES;
-- Debe mostrar: clientes, detalle_pedidos, pedidos, v_productos_basico
EOF
```

En **nodo05**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
SELECT 'productos (original)' AS tabla, COUNT(*) AS filas
FROM productos
UNION ALL
SELECT 'v_productos_detalle (frag)', COUNT(*) FROM v_productos_detalle;
SELECT COUNT(*) AS ids_solo_en_productos_original
FROM productos p
LEFT JOIN v_productos_detalle vd ON vd.id = p.id
WHERE vd.id IS NULL;
-- Debe devolver 0
DROP TABLE IF EXISTS productos;
SHOW TABLES;
-- Debe mostrar: clientes, detalle_pedidos, pedidos, v_productos_detalle
EOF
```

Verificar desde **nodo06** que las tablas Spider y VIEWS del coordinador siguen funcionando tras el DROP en los shards:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Las tablas Spider de nodo06 apuntan a v_productos_basico
y v_productos_detalle,
-- no a la tabla productos original → el DROP no las afecta
SELECT 'v_productos_basico (Spider-nodo04)' AS fuente, COUNT(*) AS filas
FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle (Spider-nodo05)', COUNT(*)
FROM v_productos_detalle
UNION ALL
SELECT 'productos (VIEW JOIN nodo04+nodo05)', COUNT(*)
FROM productos;
-- Resultado esperado: 10 / 10 / 10 – sin variación
EOF
```

E.3 Análisis estadístico de costes de transferencia (VM — nodo04, nodo05 y nodo06)

Actualizar las estadísticas del optimizador para obtener valores precisos de `TABLE_ROWS` y `AVG_ROW_LENGTH`.

En **nodo04**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Actualizar estadísticas
ANALYZE TABLE clientes;
ANALYZE TABLE pedidos;
ANALYZE TABLE detalle_pedidos;
ANALYZE TABLE v_productos_basico;
-- Tabla de costes del fragmento A
SELECT
    TABLE_NAME                AS tabla,
    TABLE_ROWS                AS filas,
    AVG_ROW_LENGTH              AS
bytes_por_fila,
    TABLE_ROWS * AVG_ROW_LENGTH AS
bytes_totales_est,
    ROUND((TABLE_ROWS * AVG_ROW_LENGTH) / 1024.0, 2) AS total_KB_est,
    'bdd-nodo04 (frag_A)'      AS ubicacion
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY bytes_totales_est DESC;
EOF
```

En **nodo05**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
ANALYZE TABLE clientes;
ANALYZE TABLE pedidos;
ANALYZE TABLE detalle_pedidos;
ANALYZE TABLE v_productos_detalle;
SELECT
    TABLE_NAME                                AS tabla,
    TABLE_ROWS                                AS filas,
    AVG_ROW_LENGTH                              AS
bytes_por_fila,
    TABLE_ROWS * AVG_ROW_LENGTH                AS
bytes_totales_est,
    ROUND((TABLE_ROWS * AVG_ROW_LENGTH) / 1024.0, 2) AS total_KB_est,
    'bdd-nodo05 (frag_B)'                      AS ubicacion
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY bytes_totales_est DESC;
EOF
```

En **nodo06**, construir la tabla de costes consolidada del laboratorio:

 Bash

```
sudo mariadb << 'EOF'
-- Tabla de referencia de costes por fragmento
-- (los valores de bytes_por_fila son referencias; reemplazar con los reales
de nodo04/05)
SELECT
    fragmento,
    tabla_origen,
    nodo,
    ip,
    filas_aprox,
    bytes_por_fila,
    filas_aprox * bytes_por_fila AS
bytes_transfer_max,
    ROUND(filas_aprox * bytes_por_fila / 1024.0, 1)
AS KB_transfer_max,
    '= coste si Spider transfiere el fragmento completo' AS nota
FROM (
    SELECT 'clientes_frag_A' AS fragmento,
           'clientes' AS tabla_origen,
           'bdd-nodo04' AS nodo,
           '192.168.56.104' AS ip,
           10 AS filas_aprox,
           120 AS bytes_por_fila
    UNION ALL SELECT 'pedidos_frag_A', 'pedidos', 'bdd-
nodo04', '192.168.56.104', 10, 90
    UNION ALL SELECT 'detalle_frag_A', 'detalle_pedidos', 'bdd-
nodo04', '192.168.56.104', 18, 60
    UNION ALL SELECT 'V_productos_basico', 'v_productos_basico', 'bdd-
nodo04', '192.168.56.104', 10, 100
    UNION ALL SELECT 'clientes_frag_B', 'clientes', 'bdd-
nodo05', '192.168.56.105', 10, 120
    UNION ALL SELECT 'pedidos_frag_B', 'pedidos', 'bdd-
nodo05', '192.168.56.105', 10, 90
    UNION ALL SELECT 'detalle_frag_B', 'detalle_pedidos', 'bdd-
nodo05', '192.168.56.105', 18, 60
    UNION ALL SELECT 'V_productos_detalle', 'v_productos_detalle', 'bdd-
nodo05', '192.168.56.105', 10, 150
) AS costes
ORDER BY nodo, bytes_transfer_max DESC;
-- Coste máximo teórico si Spider transfiere TODOS los fragmentos
SELECT 'Coste máximo total (todos los fragmentos, sin poda ni proyección)'
AS escenario,
    ROUND((10*120 + 10*90 + 18*60 + 10*100 +
```

```
        10*120 + 10*90 + 18*60 + 10*150) / 1024.0, 1)
AS total_KB;
-- Coste consulta H-1 (solo nodo04, con proyección y poda):
SELECT 'Coste H-1 (grado 0, solo nodo04, norte+este)' AS escenario,
        ROUND((10*120 + 10*90 + 18*60 + 10*100) / 1024.0, 1)
AS total_KB_H1;
EOF
```

E.4 Simulación manual de las cuatro fases de descomposición (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- CONSULTA GLOBAL ORIGINAL (emitida por el cliente en nodo06):
-- SELECT c.region, COUNT(DISTINCT c.id), COUNT(DISTINCT p.id),
--         GROUP_CONCAT(DISTINCT vb.categoria), SUM(dp.subtotal)
-- FROM clientes c
-- JOIN pedidos p ON p.cliente_id = c.id
-- JOIN detalle_pedidos dp ON dp.pedido_id = p.id
-- JOIN v_productos_basico vb ON vb.id = dp.producto_id
-- GROUP BY c.region ORDER BY facturacion_total DESC;
-- =====

-- -----
-- FASE 1: DESCOMPOSICIÓN
-- Transformaciones algebraicas aplicadas:
-- 1. Proyección temprana: solo las columnas necesarias por tabla
-- 2. Empuje de selecciones hacia las hojas
-- 3. Reordenamiento del árbol de join (más selectivo primero)
-- -----

SELECT '=== FASE 1: Descomposición ===' AS fase_actual;
SELECT c.region,
       c.id      AS c_id,
       p.id      AS p_id,
       dp.subtotal,
       dp.producto_id,
       vb.categoria
FROM   clientes      c
JOIN   pedidos        p  ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id      = dp.producto_id
ORDER  BY c.region
LIMIT 6;

-- -----
-- FASE 2: LOCALIZACIÓN
-- Sustituir relaciones globales por expresiones de fragmentos.
-- -----

SELECT '=== FASE 2: Localización ===' AS fase_actual;
-- Localización explícita de clientes
```

```

SELECT 'clientes localizado' AS rel_local,
       region,
       COUNT(*)              AS filas
FROM (
    SELECT region FROM clientes WHERE region IN ('norte','este')
    UNION ALL
    SELECT region FROM clientes WHERE region IN ('sur','oeste')
) AS clientes_localizados
GROUP BY region ORDER BY region;
-- Simplificación con predicado: region='norte' elimina frag_B
SELECT 'Simplificación con WHERE region=norte (frag_B vacío → eliminado)'
AS demo;
SELECT 'frag_A (norte+este)' AS fragmento,
       COUNT(*)              AS filas_relevantes
FROM   clientes WHERE region IN ('norte','este')
UNION ALL
SELECT 'frag_B (sur+oeste) - ELIMINADO por predicado',
       COUNT(*) AS filas_relevantes_cero
FROM   clientes WHERE region IN ('sur','oeste') AND region = 'norte';

-- -----
-- FASE 3: OPTIMIZACIÓN GLOBAL
-- Sub-agregación por shard para reducir filas devueltas al coordinador
-- -----

SELECT '=== FASE 3: Optimización global (sub-agregación por shard) ==='
AS fase_actual;
-- Sub-resultado de frag_A (lo que Spider recibiría de nodo04):
SELECT c.region,
       COUNT(DISTINCT c.id)              AS
clientes_activos,
       COUNT(DISTINCT p.id)              AS pedidos,
       GROUP_CONCAT(DISTINCT vb.categoria ORDER BY vb.categoria)
AS categorias,
       ROUND(SUM(dp.subtotal), 2)        AS
facturacion
FROM   clientes          c
JOIN   pedidos            p ON p.cliente_id = c.id
JOIN   spider_detalle_nodo04 dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id        = dp.producto_id
WHERE  c.region IN ('norte','este')
GROUP BY c.region;
-- Sub-resultado de frag_B (lo que Spider recibiría de nodo05):
SELECT c.region,
       COUNT(DISTINCT c.id)              AS
clientes_activos,

```

```

        COUNT(DISTINCT p.id)                                AS pedidos,
        GROUP_CONCAT(DISTINCT vb.categoria ORDER BY vb.categoria)
AS categorias,
        ROUND(SUM(dp.subtotal), 2)                        AS
facturacion
FROM    clientes                c
JOIN    pedidos                p  ON  p.cliente_id = c.id
JOIN    spider_detalle_nodo05 dp ON dp.pedido_id  = p.id
JOIN    v_productos_basico     vb ON vb.id         = dp.producto_id
WHERE   c.region IN ('sur', 'oeste')
GROUP   BY c.region;

-- -----
-- FASE 4: EJECUCIÓN DISTRIBUIDA
-- El coordinador recibe resultados parciales y ensambla el resultado final
-- -----

SELECT '=== FASE 4: Ejecución distribuida (ensamblado en el coordinador) ==='
AS fase_actual;
SELECT region,
        SUM(clientes_activos)
AS clientes_activos,
        SUM(pedidos)
AS pedidos,
        GROUP_CONCAT(DISTINCT categorias ORDER BY categorias SEPARATOR ', ')
AS categorias,
        ROUND(SUM(facturacion), 2)
AS facturacion_total
FROM (
        SELECT c.region,
                COUNT(DISTINCT c.id)                                AS
clientes_activos,
                COUNT(DISTINCT p.id)                                AS
pedidos,
                GROUP_CONCAT(DISTINCT vb.categoria ORDER BY vb.categoria)
AS categorias,
                ROUND(SUM(dp.subtotal), 2)                        AS
facturacion
        FROM    clientes                c
        JOIN    pedidos                p  ON  p.cliente_id = c.id
        JOIN    spider_detalle_nodo04 dp ON dp.pedido_id  = p.id
        JOIN    v_productos_basico     vb ON vb.id         = dp.producto_id
        WHERE   c.region IN ('norte', 'este')
        GROUP   BY c.region
        UNION ALL
        SELECT c.region,

```

```

        COUNT(DISTINCT c.id),
        COUNT(DISTINCT p.id),
        GROUP_CONCAT(DISTINCT vb.categoria ORDER BY vb.categoria),
        ROUND(SUM(dp.subtotal), 2)
FROM    clientes            c
JOIN    pedidos             p  ON  p.cliente_id = c.id
JOIN    spider_detalle_nodo05 dp ON dp.pedido_id = p.id
JOIN    v_productos_basico   vb ON vb.id        = dp.producto_id
WHERE   c.region IN ('sur', 'oeste')
GROUP   BY c.region
) AS resultados_parciales
GROUP   BY region
ORDER   BY facturacion_total DESC;

-- -----
-- VERIFICACIÓN: el resultado de Fase 4 debe ser idéntico a H-6 directa
-- -----
SELECT '=== Referencia: consulta H-6 directa via Spider ===' AS referencia;
SELECT c.region,
        COUNT(DISTINCT c.id)                                AS
clientes_activos,
        COUNT(DISTINCT p.id)                                AS
pedidos,
        GROUP_CONCAT(DISTINCT vb.categoria ORDER BY vb.categoria)
AS categorias,
        ROUND(SUM(dp.subtotal), 2)                            AS
facturacion_total
FROM    clientes            c
JOIN    pedidos             p  ON  p.cliente_id = c.id
JOIN    detalle_pedidos     dp ON dp.pedido_id = p.id
JOIN    v_productos_basico   vb ON vb.id        = dp.producto_id
GROUP   BY c.region
ORDER   BY facturacion_total DESC;
EOF

```

E.5 Implementación manual de semijoin distribuida (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- ESCENARIO: JOIN detalle_pedidos ⋈ v_productos_basico
-- =====

-- -----
-- ESTRATEGIA A: JOIN NAIVE (sin reducción semijoin)
-- -----

SELECT '=== Estrategia A: JOIN naive (sin semijoin) ===' AS estrategia;
FLUSH STATUS;
SELECT dp.pedido_id,
       vb.nombre      AS producto,
       vb.categoria,
       dp.cantidad,
       dp.subtotal
FROM   detalle_pedidos dp
JOIN   v_productos_basico vb ON vb.id = dp.producto_id
ORDER BY dp.pedido_id
LIMIT 10;
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next',
    'Handler_read_key',
    'Handler_read_next',
    'Bytes_sent'
);

-- -----
-- ESTRATEGIA B: SEMIJOIN MANUAL (algoritmo Bernstein-Chiu)
-- -----

SELECT '=== Estrategia B: Semijoin manual (Bernstein-Chiu) ==='
AS estrategia;
-- Paso B.1: Proyectar los IDs de join desde detalle_pedidos
SELECT '--- Paso B.1: IDs únicos de join en detalle_pedidos ---' AS paso;
SELECT DISTINCT producto_id AS id_join
FROM   detalle_pedidos
ORDER BY producto_id;
-- Paso B.2: Filtrar v_productos_basico (semijoin aplicado en nodo04)
SELECT '--- Paso B.2: v_productos_basico reducido por semijoin ---' AS paso;
SELECT id, nombre, categoria, precio
FROM   v_productos_basico
WHERE  id IN (
```

```

        SELECT DISTINCT producto_id FROM detalle_pedidos
    );
-- Paso B.3: JOIN completo usando el resultado del semijoin
SELECT '--- Paso B.3: JOIN final con v_productos_basico filtrado ---'
AS paso;
FLUSH STATUS;
SELECT dp.pedido_id,
       vb.nombre    AS producto,
       vb.categoria,
       dp.cantidad,
       dp.subtotal
FROM   detalle_pedidos    dp
JOIN   v_productos_basico vb ON vb.id = dp.producto_id
      AND vb.id IN (
          SELECT DISTINCT producto_id
          FROM   detalle_pedidos
      )

ORDER BY dp.pedido_id
LIMIT 10;
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next',
    'Handler_read_key',
    'Handler_read_next',
    'Bytes_sent'
);

-- -----
-- ANÁLISIS DEL BENEFICIO DEL SEMIJOIN
-- -----

SELECT '=== Análisis del beneficio teórico ===' AS analisis;
SELECT
    'Paso B.1: transferir solo IDs de join'           AS etapa,
    COUNT(DISTINCT producto_id)                       AS filas_transferidas,
    COUNT(DISTINCT producto_id) * 4                   AS bytes_aprox,
    'IDs (4 bytes cada uno)'                          AS que_se_transfiere
FROM detalle_pedidos
UNION ALL
SELECT
    'Estrategia A: v_productos_basico completo',
    COUNT(*),
    COUNT(*) * 100,
    'Filas completas de v_productos_basico'
FROM v_productos_basico
UNION ALL
SELECT

```

```
'Paso B.2: v_productos_basico reducido (semijoin)',  
COUNT(*),  
COUNT(*) * 100,  
'Solo filas con producto_id en detalle_pedidos'  
FROM v_productos_basico  
WHERE id IN (SELECT DISTINCT producto_id FROM detalle_pedidos);  
-- Fórmula general del beneficio del semijoin:  
SELECT 'Condición de rentabilidad del semijoin:' AS formula,  
       'size(S) > size( $\pi_a$ (S)) + size(R  $\times$  S)' AS condicion,  
       'Con nuestros datos: 1 KB > 0.04 KB + 1 KB  $\rightarrow$  NO es rentable (caso  
límite)' AS evaluacion;  
EOF
```

E.6 Comparación: proyección temprana vs. `SELECT *` (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Versión 1: SELECT * (todas las columnas de clientes)
SELECT '=== Versión 1: SELECT * FROM clientes (todas las columnas) ==='
AS experimento;
FLUSH STATUS;
SELECT * FROM clientes WHERE region = 'norte';
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next', 'Handler_read_key',
    'Handler_read_next', 'Bytes_sent'
);

-- Versión 2: Proyección explícita (solo 4 columnas)
SELECT '=== Versión 2: Proyección explícita (id, nombre, apellido, region)
=== ' AS experimento;
FLUSH STATUS;
SELECT id, nombre, apellido, region
FROM   clientes
WHERE  region = 'norte';
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next', 'Handler_read_key',
    'Handler_read_next', 'Bytes_sent'
);

-- Resumen: columnas de clientes y su ancho estimado
SELECT COLUMN_NAME,
       DATA_TYPE,
       COALESCE(CHARACTER_MAXIMUM_LENGTH, NUMERIC_PRECISION, 8)
AS ancho_max_bytes,
       CASE
           WHEN COLUMN_NAME IN ('id', 'nombre', 'apellido', 'region')
THEN 'PROYECTADA'
           ELSE 'omitida con proyección'
       END AS en_version2
FROM   information_schema.COLUMNS
WHERE  TABLE_SCHEMA = 'lab_bdd'
      AND TABLE_NAME  = 'clientes'
ORDER BY ORDINAL_POSITION;
EOF
```

E.7 Verificación de poda por índice en subconsultas remotas (VM — nodo06 y nodo04)

 Bash

```
# En nodo06: plan de ejecución del coordinador
sudo mariadb lab_bdd << 'EOF'
SELECT '=== A1: clientes SIN filtro (ambas particiones) ===' AS variante;
EXPLAIN SELECT id, nombre, region FROM clientes;
SELECT '=== A2: clientes CON region=norte (solo frag_A) ===' AS variante;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region = 'norte';
SELECT '=== A3: clientes CON region IN (norte,este) (solo frag_A) ==='
AS variante;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region IN
('norte', 'este');
SELECT '=== A4: clientes CON region IN (norte,sur) (frag_A + frag_B) ==='
AS variante;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region IN
('norte', 'sur');
SELECT '=== B1: pedidos CON region=sur+oeste (solo frag_B) ===' AS variante;
EXPLAIN SELECT id, total FROM pedidos WHERE region IN ('sur', 'oeste');
SELECT '=== C1: detalle_pedidos VIEW (siempre accede a ambas tablas Spider)
=== ' AS variante;
EXPLAIN SELECT COUNT(*) FROM detalle_pedidos;
SELECT '=== C2: spider_detalle_nodo04 directa (solo nodo04) ===' AS variante;
EXPLAIN SELECT COUNT(*) FROM spider_detalle_nodo04;
SELECT '=== D1: SELECT id FROM v_productos_basico (solo nodo04) ==='
AS variante;
EXPLAIN SELECT id, nombre FROM v_productos_basico WHERE id = 1;
SELECT '=== D2: SELECT * FROM productos VIEW (JOIN nodo04+nodo05) ==='
AS variante;
EXPLAIN SELECT id, nombre, descripcion FROM productos WHERE id = 1;
EOF
```

 Bash

```
# En nodo04: subconsultas remotas simuladas y verificación de planes locales
sudo mariadb lab_bdd << 'EOF'
SELECT '=== Subconsulta local: clientes WHERE region=norte ==='
AS subconsulta;
EXPLAIN
SELECT id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  region = 'norte';
SELECT '=== Subconsulta local: pedidos WHERE region=norte ==='
AS subconsulta;
EXPLAIN
SELECT id, cliente_id, total, estado
FROM   pedidos
WHERE  region = 'norte';
SELECT '=== Subconsulta local: detalle_pedidos con IN (IDs de pedidos) ==='
AS subconsulta;
EXPLAIN
SELECT id, pedido_id, producto_id, cantidad, subtotal
FROM   detalle_pedidos
WHERE  pedido_id IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
-- Medir el I/O real con predicado de región (usa índice) vs. sin él
SELECT '=== I/O real: pedidos WHERE region=norte ===' AS medicion;
FLUSH STATUS;
SELECT id, cliente_id, total FROM pedidos WHERE region = 'norte';
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next', 'Handler_read_key', 'Handler_read_next'
);
SELECT '=== I/O real: pedidos SIN predicado (full fragment scan) ==='
AS medicion;
FLUSH STATUS;
SELECT id, cliente_id, total FROM pedidos;
SHOW STATUS WHERE Variable_name IN (
    'Handler_read_rnd_next', 'Handler_read_key', 'Handler_read_next'
);
EOF
```

Observación pedagógica: con fragmentos de 10 filas, InnoDB puede preferir un full scan sobre el uso del índice porque el coste de acceso a la estructura del índice supera el de leer directamente las 10 filas del tablespace. Este comportamiento es correcto y esperado. Con fragmentos de 10 000 filas o más, el optimizador elegiría el índice y el beneficio sería proporcional a la selectividad del predicado.

E.8 Crear el procedimiento `analizar_consulta()` (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
DROP PROCEDURE IF EXISTS analizar_consulta;
DELIMITER //
CREATE PROCEDURE analizar_consulta(
    IN p_region          ENUM('norte', 'sur', 'este', 'oeste'),
    IN p_incluir_vertical BOOLEAN
)
COMMENT 'Muestra el plan de descomposición estimado y el grado de localidad'
BEGIN
    DECLARE v_nodo_h      VARCHAR(30)  DEFAULT 'nodo04+nodo05 (ambos)';
    DECLARE v_filas_h     INT           DEFAULT 20;
    DECLARE v_filas_dp    INT           DEFAULT 36;
    DECLARE v_grado       TINYINT       DEFAULT 1;
    DECLARE v_grado_txt   VARCHAR(80)   DEFAULT 'Horizontal puro -
ambos shards';
    IF p_region IN ('norte', 'este') THEN
        SET v_nodo_h     = 'bdd-nodo04 (frag_A)';
        SET v_filas_h     = 10;
        SET v_filas_dp    = 18;
        SET v_grado       = IF(p_incluir_vertical, 1, 0);
    ELSEIF p_region IN ('sur', 'oeste') THEN
        SET v_nodo_h     = 'bdd-nodo05 (frag_B)';
        SET v_filas_h     = 10;
        SET v_filas_dp    = 18;
        SET v_grado       = IF(p_incluir_vertical, 1, 0);
    ELSE
        SET v_grado = IF(p_incluir_vertical, 2, 1);
    END IF;
    SET v_grado_txt = CASE v_grado
        WHEN 0 THEN 'Máxima localidad - acceso a un solo nodo físico'
        WHEN 1 THEN CONCAT(IF(p_region IS NULL,
                                'H-puro: ambos shards horizontales',
                                'H+V-parcial: shard horizontal único + nodo05
para detalle'))
        WHEN 2 THEN 'Distribución total - H (nodo04+nodo05) y
V (nodo04+nodo05)'
        ELSE '?'
    END;
    SELECT CONCAT(
        'PLAN DE DESCOMPOSICIÓN ESTIMADO',
        ' | Región: ', COALESCE(p_region, 'TODAS'),
```

```

        ' | Vertical: ', IF(p_incluir_vertical, 'Sí
(v_productos_detalle)', 'NO'),
        ' | Grado de localidad: ', v_grado, ' - ', v_grado_txt
    ) AS descripcion_plan;
SELECT fragmento,
       nodo_fisico,
       filas_est,
       bytes_por_fila,
       filas_est * bytes_por_fila AS
bytes_transfer_est,
       ROUND(filas_est * bytes_por_fila / 1024.0, 2) AS KB_est
FROM (
    SELECT 'clientes' AS fragmento,
           v_nodo_h AS nodo_fisico,
           v_filas_h AS filas_est,
           120 AS bytes_por_fila, 1 AS ord
    UNION ALL
    SELECT 'pedidos', v_nodo_h, v_filas_h, 90, 2
    UNION ALL
    SELECT 'detalle_pedidos', v_nodo_h, v_filas_dp, 60, 3
    UNION ALL
    SELECT 'v_productos_basico', 'bdd-nodo04 (siempre)', 10, 100, 4
    UNION ALL
    SELECT 'v_productos_detalle', 'bdd-nodo05 (siempre)',
           IF(p_incluir_vertical, 10, 0),
           IF(p_incluir_vertical, 150, 0), 5
    ) AS plan
WHERE filas_est > 0
ORDER BY ord;
SELECT ROUND(
    (v_filas_h * 120 +
    v_filas_h * 90 +
    v_filas_dp * 60 +
    10 * 100 +
    IF(p_incluir_vertical, 10 * 150, 0)) / 1024.0, 1
    ) AS total_KB_estimado,
    v_grado AS grado_de_localidad,
    v_grado_txt AS descripcion;
END //
DELIMITER ;
-- Pruebas del procedimiento
SELECT '--- analizar_consulta(norte, FALSE) → grado 0, solo nodo04 ---'
AS demo;
CALL analizar_consulta('norte', FALSE);
SELECT '--- analizar_consulta(norte, TRUE) → grado 1, nodo04+nodo05

```

```

(vertical) ---' AS demo;
CALL analizar_consulta('norte', TRUE);
SELECT '--- analizar_consulta(NULL, FALSE) → grado 1, ambos shards
horizontales ---' AS demo;
CALL analizar_consulta(NULL, FALSE);
SELECT '--- analizar_consulta(NULL, TRUE) → grado 2, distribución total ---'
AS demo;
CALL analizar_consulta(NULL, TRUE);
SELECT '--- analizar_consulta(sur, FALSE) → grado 0, solo nodo05 ---'
AS demo;
CALL analizar_consulta('sur', FALSE);
EOF

```

E.9 Crear `bdd-nodo07` (`bdd-cliente`) como clon enlazado (host)

 PowerShell

```

# Verificar que todos los nodos están apagados antes de clonar
VBoxManage list runningvms
# La salida debe estar vacía
# Crear bdd-nodo07 como clon enlazado desde fase06-completa de nodo01
# fase06-completa = Ubuntu Server + SSH + IP configurada + SIN MariaDB Server
VBoxManage clonevm "bdd-nodo01" `
    --snapshot      "fase06-completa" `
    --options        linked `
    --name           "bdd-nodo07" `
    --basefolder     "C:\LabBDD\VMs" `
    --register
VBoxManage modifyvm "bdd-nodo07" `
    --memory         1024 `
    --description     "CLIENTE EXTERNO (bdd-cliente). Solo mariadb-client. IP
192.168.56.107. Conecta exclusivamente a nodo06. Fase 16."
# Verificar el registro
VBoxManage showvminfo "bdd-nodo07" | findstr /I "Name State Memory Description"

```

E.10 Configurar `bdd-nodo07` (hostname, IP y cliente MariaDB) (host + VM)

PowerShell

```
# Arrancar solo nodo07 (nodo01 apagado → sin conflicto de IP)
VBoxManage startvm "bdd-nodo07" --type headless
Start-Sleep -Seconds 35
# El clon hereda temporalmente la IP de nodo01 (192.168.56.101)
ssh bddadmin@192.168.56.101
```

(VM — nodo07):

Bash

```
# — 1. Cambiar el hostname —————
sudo hostnamectl set-hostname bdd-cliente
sudo sed -i 's/bdd-nodo01/bdd-cliente/g' /etc/hosts
hostname
# Esperado: bdd-cliente
```

Bash

```
# — 2. Cambiar la IP estática de .101 a .107 —————
NETPLAN_FILE=$(ls /etc/netplan/*.yaml | head -1)
echo "Archivo Netplan: $NETPLAN_FILE"
sudo sed -i 's/192\.\168\.\56\.\101/192.168.56.107/g' "$NETPLAN_FILE"
grep "192.168" "$NETPLAN_FILE"
# Aplicar — la sesión SSH se interrumpe aquí
sudo netplan apply
```

Reconectar a la nueva IP:

PowerShell

```
Start-Sleep -Seconds 8
ssh bddadmin@192.168.56.107
```

(VM — nodo07) — continuar:

 Bash

```
# — 3. Verificar la nueva IP —————  
ip addr show | grep "192.168.56"  
# Esperado: inet 192.168.56.107/24  
ping -c 2 192.168.56.1
```

 Bash

```
# — 4. Regenerar las claves SSH —————  
sudo rm -f /etc/ssh/ssh_host_*  
sudo ssh-keygen -A  
sudo systemctl restart ssh  
echo "Claves SSH regeneradas:"  
ls /etc/ssh/ssh_host_*.pub
```

 Bash

```
# — 5. Instalar mariadb-client (SIN el servidor) —————  
sudo apt-get update  
sudo apt-get install -y mariadb-client  
# Verificar que el cliente existe pero el servidor NO  
which mariadb  
# Esperado: /usr/bin/mariadb  
systemctl status mariadb 2>&1 | head -5  
# Esperado: Unit mariadb.service could not be found
```

 Bash

```
# — 6. Registrar todos los nodos en /etc/hosts —————
sudo tee -a /etc/hosts > /dev/null << 'EOF'
# Laboratorio BDD – todos los nodos
192.168.56.101 bdd-nodo01
192.168.56.102 bdd-nodo02
192.168.56.103 bdd-nodo03
192.168.56.104 bdd-nodo04
192.168.56.105 bdd-nodo05
192.168.56.106 bdd-nodo06
192.168.56.107 bdd-cliente
EOF
cat /etc/hosts
```

 Bash

```
# — 7. Verificar conectividad hacia el coordinador —————
ping -c 3 192.168.56.106
nc -zv 192.168.56.106 3306
# Esperado: Connection to 192.168.56.106 3306 port [tcp/mysql] succeeded!
```

E.11 Reiniciar nodo04, nodo05 y nodo06 con nodo07 ya activo (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo04" --type headless
VBoxManage startvm "bdd-nodo05" --type headless
VBoxManage startvm "bdd-nodo06" --type headless
Start-Sleep -Seconds 40
VBoxManage list runningvms
# Deben aparecer: bdd-nodo04, bdd-nodo05, bdd-nodo06, bdd-nodo07
```

Verificar la conectividad de red completa desde `bdd-cliente` :

 Bash

```
# En bdd-nodo07 - verificar toda la red del laboratorio
for ip in 104 105 106; do
    echo -n "Conectividad a 192.168.56.$ip: "
    ping -c 1 -W 2 192.168.56.$ip > /dev/null 2>&1 && echo "OK" ||
echo "FALLA"
done
# Verificar acceso al puerto 3306 del coordinador
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT 'Conexion exitosa a nodo06' AS estado, @@hostname AS
coordinador;" \
    2>/dev/null
```

Verificar que `bdd-cliente` NO puede conectarse directamente a los shards:

 Bash

```
# Intento de acceso directo a nodo04 (debe fallar)
mariadb -h 192.168.56.104 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT 1;" \
    2>&1 | head -3
# Esperado: ERROR 1045 (28000): Access denied
# Intento de acceso directo a nodo05 (debe fallar)
mariadb -h 192.168.56.105 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT 1;" \
    2>&1 | head -3
# Esperado: ERROR 1045 (28000): Access denied
```

E.12 Crear el usuario `app_final` para `bdd-cliente` en nodo06 (VM — nodo06)

 Bash

```
sudo mariadb << 'EOF'
-- Verificar entradas actuales de app_final
SELECT User, Host FROM mysql.user WHERE User = 'app_final';
-- Crear entrada específica para bdd-cliente (192.168.56.107)
CREATE USER IF NOT EXISTS 'app_final'@'192.168.56.107'
    IDENTIFIED BY 'AppFinal_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*
    TO 'app_final'@'192.168.56.107';
GRANT EXECUTE ON lab_bdd.*
    TO 'app_final'@'192.168.56.107';
FLUSH PRIVILEGES;
-- Verificar todos los hosts de app_final
SELECT User, Host FROM mysql.user WHERE User = 'app_final' ORDER BY Host;
SHOW GRANTS FOR 'app_final'@'192.168.56.107';
EOF
```

E.13 Pruebas de transparencia desde `bdd-cliente` (VM — nodo07)

 Bash

```
echo "=====
echo "PRUEBA 1: Conexión al sistema distribuido"
echo "=====
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    lab_bdd \
    -e "SELECT @@hostname AS servidor_conectado,
        @@server_id AS server_id,
        DATABASE() AS base_de_datos;" \
    2>/dev/null
# Esperado: bdd-nodo06, 6, lab_bdd
```

 Bash

```
echo "===== "  
echo "PRUEBA 2: Ver objetos del esquema"  
echo "===== "  
mariadb -h 192.168.56.106 \  
    -u app_final \  
    -p'AppFinal_2025!' \  
    lab_bdd \  
    -e "SHOW TABLES;" \  
    2>/dev/null
```

 Bash

```
echo "===== "  
echo "PRUEBA 3: Lectura global (accede a 2 shards invisiblemente)"  
echo "===== "  
mariadb -h 192.168.56.106 \  
    -u app_final \  
    -p'AppFinal_2025!' \  
    lab_bdd \  
    -e "SELECT region,  
        COUNT(*) AS total_clientes  
        FROM clientes  
        GROUP BY region  
        ORDER BY region;" \  
    2>/dev/null  
# Esperado: este=5, norte=5, oeste=5, sur=5
```

 Bash

```
echo "=====
echo "PRUEBA 4: Poda automática (accede a solo un shard)"
echo "=====
mariadb -h 192.168.56.106 \
-u app_final \
-p'AppFinal_2025!' \
lab_bdd \
-e "SELECT id, nombre, apellido, ciudad
    FROM clientes
    WHERE region = 'norte'
    ORDER BY id;" \
2>/dev/null
# Esperado: 5 clientes de la región norte
```

 Bash

```
echo "=====
echo "PRUEBA 5: Catálogo de productos (JOIN distribuido vertical)"
echo "=====
mariadb -h 192.168.56.106 \
-u app_final \
-p'AppFinal_2025!' \
lab_bdd \
-e "SELECT id, sku, nombre, categoria, precio, stock,
    LEFT(descripcion, 60) AS descripcion_preview,
    peso_kg
    FROM productos
    ORDER BY categoria, precio;" \
2>/dev/null
# Esperado: 10 productos con columnas de ambos fragmentos verticales
```

 Bash

```
echo "=====
echo "PRUEBA 6: Reporte de negocio vía vista encapsulada"
echo "=====
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    lab_bdd \
    -e "SELECT region,
        COUNT(*) AS lineas_facturadas,
        ROUND(SUM(subtotal), 2) AS total_region
    FROM   reporte_pedidos_detallado
    GROUP  BY region
    ORDER  BY total_region DESC;" \
    2>/dev/null
# Esperado: 4 filas con datos de facturación de todas las regiones
```

 Bash

```
echo "=====
echo "PRUEBA 7: Llamada al procedimiento almacenado"
echo "=====
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    lab_bdd \
    -e "CALL consulta_regional(NULL);" \
    2>/dev/null
# Esperado: 4 filas con resumen de todas las regiones
echo "--- Procedimiento filtrado por región ---"
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    lab_bdd \
    -e "CALL consulta_regional('este');" \
    2>/dev/null
# Esperado: 1 fila con datos de la región este
```

 Bash

```
echo "=====
echo "PRUEBA 8: Verificación de transparencia"
echo "El cliente NO puede ver la infraestructura interna"
echo "=====
echo "--- 8a: Intentar SELECT en mysql.servers (debe fallar) ---"
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT * FROM mysql.servers;" \
    2>&1 | head -3
# Esperado: ERROR 1142 (42000): SELECT command denied
echo "--- 8b: Intentar conexión directa a nodo04 (debe fallar) ---"
mariadb -h 192.168.56.104 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT 1;" \
    2>&1 | head -3
# Esperado: ERROR 1045 (28000): Access denied
echo "--- 8c: Intentar conexión directa a nodo05 (debe fallar) ---"
mariadb -h 192.168.56.105 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT 1;" \
    2>&1 | head -3
# Esperado: ERROR 1045 (28000): Access denied
echo "--- 8d: Intentar ver usuarios del sistema en nodo06 (debe fallar) ---"
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    -e "SELECT User, Host FROM mysql.user;" \
    2>&1 | head -3
# Esperado: ERROR 1142 (42000): SELECT command denied
echo ""
echo "=====
echo "RESUMEN: conteos globales desde bdd-cliente"
echo "=====
mariadb -h 192.168.56.106 \
    -u app_final \
    -p'AppFinal_2025!' \
    lab_bdd \
    -e "SELECT 'clientes' AS tabla, COUNT(*) AS filas
FROM clientes
      UNION ALL
```

```
SELECT 'pedidos',          COUNT(*)    FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)    FROM detalle_pedidos
UNION ALL
SELECT 'productos (VIEW)', COUNT(*)    FROM productos;" \
2>/dev/null
# Esperado: 20 / 20 / ~36 / 10
```

E.14 Apagar los nodos y tomar los snapshots **fase16-completa** (host)

 Bash

```
# En la sesión SSH de bdd-nodo07 (bdd-cliente)
sudo poweroff
# En la sesión SSH de bdd-nodo06
sudo poweroff
# En la sesión SSH de bdd-nodo04
sudo poweroff
# En la sesión SSH de bdd-nodo05
sudo poweroff
```

 PowerShell

```
Start-Sleep -Seconds 35
VBoxManage list runningvms
# La salida debe estar vacía
```

PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase16-completa" `
--description "Sin cambios en Fase 16. Miembro Galera. Snapshot de hito
de fase."
VBoxManage snapshot "bdd-nodo02" take "fase16-completa" `
--description "Sin cambios en Fase 16. Miembro Galera. Snapshot de hito
de fase."
VBoxManage snapshot "bdd-nodo03" take "fase16-completa" `
--description "Sin cambios en Fase 16. Miembro Galera. Snapshot de hito
de fase."
VBoxManage snapshot "bdd-nodo04" take "fase16-completa" `
--description "SHARD-A: tabla productos original eliminada.
v_productos_basico(10 filas) es el fragmento vertical definitivo. Analisis de
costes ejecutado. Poda por indice verificada con EXPLAIN."
VBoxManage snapshot "bdd-nodo05" take "fase16-completa" `
--description "SHARD-B: tabla productos original eliminada.
v_productos_detalle(10 filas) es el fragmento vertical definitivo. Analisis de
costes ejecutado."
VBoxManage snapshot "bdd-nodo06" take "fase16-completa" `
--description "COORDINADOR: PROCEDURE analizar_consulta() creado.
app_final@192.168.56.107 habilitado. 4 fases de descomposicion simuladas.
Semijoin manual verificado. Transparencia verificada desde bdd-cliente."
VBoxManage snapshot "bdd-nodo07" take "fase16-completa" `
--description "CLIENTE EXTERNO (bdd-cliente): primer snapshot. Ubuntu Server +
mariadb-client. IP 192.168.56.107. Conecta a nodo06 via app_final. 8 pruebas de
transparencia superadas. Sin acceso a shards internos."
```

PowerShell

```
VBoxManage snapshot "bdd-nodo04" list
VBoxManage snapshot "bdd-nodo05" list
VBoxManage snapshot "bdd-nodo06" list
VBoxManage snapshot "bdd-nodo07" list
```

F. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes:

1. `SHOW TABLES IN lab_bdd` en nodo04 **no muestra** la tabla `productos` (solo `clientes`, `detalle_pedidos`, `pedidos`, `v_productos_basico`).

2. `SHOW TABLES IN lab_bdd` en nodo05 **no muestra** la tabla `productos` (solo `clientes`, `detalle_pedidos`, `pedidos`, `v_productos_detalle`).
3. Las VIEWS `v_productos_basico`, `v_productos_detalle` y `productos` en nodo06 devuelven 10 filas cada una tras eliminar la tabla `productos` original de los shards.
4. `ANALYZE TABLE` se ejecutó correctamente en las cuatro tablas de cada shard; `information_schema.TABLES` muestra `TABLE_ROWS > 0` y `AVG_ROW_LENGTH > 0` para todas ellas.
5. La simulación de la **Fase 4 (ejecución distribuida manual)** del bloque E.4 devuelve exactamente **4 filas** con los mismos valores que la consulta H-6 directa vía Spider; los resultados son idénticos en cada columna.
6. La **estrategia A (JOIN naive)** y la **estrategia B (semijoin manual)** del bloque E.5 devuelven el mismo resultado de negocio (mismas filas y mismos valores).
7. `SHOW STATUS LIKE 'Handler_read_rnd_next'` devuelve valores > 0 tras ejecutar una consulta de conteo sin índice sobre una tabla de un shard (comportamiento esperado con fragmentos pequeños).
8. `EXPLAIN SELECT id, nombre FROM clientes WHERE region = 'norte'` en nodo06 muestra `partitions: frag_A` únicamente.
9. `EXPLAIN SELECT id, nombre FROM clientes` sin filtro en nodo06 muestra ambas particiones `frag_A, frag_B`.
10. `EXPLAIN SELECT COUNT(*) FROM detalle_pedidos` en nodo06 no muestra poda de partición (VIEW UNION ALL accede a ambas tablas subyacentes).
11. `EXPLAIN SELECT id, pedido_id FROM detalle_pedidos WHERE pedido_id IN (1,2,3)` ejecutado directamente en nodo04 muestra uso del índice `idx_pedido`.
12. La comparación de `Bytes_sent` entre `SELECT *` y proyección explícita sobre `clientes WHERE region='norte'` muestra un valor mayor para `SELECT *`.
13. El procedimiento `analizar_consulta('norte', FALSE)` devuelve `grado_de_localidad = 0` y los fragmentos de nodo04 únicamente.
14. El procedimiento `analizar_consulta(NULL, TRUE)` devuelve `grado_de_localidad = 2`.
15. `VBoxManage showvminfo "bdd-nodo07"` confirma la VM registrada con el nombre correcto.
16. `hostname` en nodo07 devuelve `bdd-cliente`.
17. `ip addr show | grep 192.168.56` en nodo07 muestra `192.168.56.107/24`.
18. `systemctl status mariadb` en nodo07 indica que el servicio no existe (cliente solamente).
19. `which mariadb` en nodo07 devuelve `/usr/bin/mariadb`.
20. La **Prueba 3** desde bdd-cliente devuelve exactamente 4 filas con 5 clientes por región.
21. La **Prueba 4** desde bdd-cliente devuelve exactamente 5 clientes con `region = 'norte'`.

22. La **Prueba 5** desde bdd-cliente devuelve exactamente 10 productos con columnas de ambos fragmentos verticales combinadas.
 23. La **Prueba 6** desde bdd-cliente devuelve 4 filas con datos de facturación no nulos.
 24. `CALL consulta_regional(NULL)` desde bdd-cliente devuelve **4 filas**.
 25. `CALL consulta_regional('este')` desde bdd-cliente devuelve **1 fila**.
 26. La **Prueba 8a** produce `ERROR 1142 (42000)` o similar (acceso denegado a `mysql.servers`).
 27. Las **Pruebas 8b y 8c** producen `ERROR 1045 (28000)` al intentar conectar a nodo04 y nodo05.
 28. Los snapshots `fase16-completa` existen en los **siete nodos** del laboratorio.
 29. El estudiante puede enumerar de memoria las cuatro fases del procesamiento de consultas distribuidas.
 30. El estudiante puede definir el semijoin distribuido y la condición que lo hace rentable.
-

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>SELECT</code> <code>COUNT(*) FROM</code> <code>productos</code> en nodo06 falla después de <code>DROP TABLE</code> <code>productos</code> en los shards	La tabla Spider <code>v_productos_ba</code> <code>sico</code> de nodo06 apunta a la tabla correcta pero la VIEW <code>productos</code> hace JOIN de <code>v_productos_ba</code> <code>sico</code> y <code>v_productos_de</code> <code>talle</code>; si alguna de éstas se eliminó, Spider no encuentra la tabla remota	Verificar en nodo04: <code>SHOW</code> <code>TABLES LIKE</code> <code>'v_productos%</code> <code>'</code> — debe mostrar <code>v_productos_ba</code> <code>sico</code>; si no existe, restaurar nodo04 desde <code>fase14-</code> <code>completa</code>
<code>SHOW TABLES IN</code> <code>lab_bdd</code> en nodo04 sigue mostrando <code>productos</code> tras el <code>DROP TABLE</code>	El <code>DROP TABLE</code> no se ejecutó o se ejecutó en el nodo equivocado	Ejecutar <code>DROP</code> <code>TABLE IF</code> <code>EXISTS</code> <code>lab_bdd.produc</code> <code>tos</code> directamente en nodo04; verificar con <code>SHOW TABLES</code>
La Fase 4 (ejecución manual) produce resultados distintos a H-6	Las tablas Spider directas <code>spider_detalle</code> <code>_nodo04</code> y <code>spider_detalle</code> <code>_nodo05</code> tienen datos distintos a los que devuelve la VIEW <code>detalle_pedido</code> <code>s</code>	Verificar con <code>SELECT</code> <code>COUNT(*) FROM</code> <code>spider_detalle</code> <code>_nodo04</code> y <code>SELECT</code> <code>COUNT(*) FROM</code> <code>spider_detalle</code> <code>_nodo05</code> desde nodo06; la suma debe igualar <code>SELECT</code>

		<pre>COUNT(*) FROM detalle_pedido s</pre>
<pre>nc -zv 192.168.56.106 3306 desde bdd-cliente devuelve Connection refused</pre>	MariaDB en nodo06 tiene <pre>bind-address = 127.0.0.1</pre> o no está activo	En nodo06: <pre>sudo mariadb - e "SHOW VARIABLES LIKE 'bind_address' ;"</pre> — si devuelve <pre>127.0.0.1 ,</pre> aplicar el <code>sed</code> de la Fase 14; verificar con <pre>sudo systemctl status mariadb</pre>
<pre>ssh bddadmin@192.1 68.56.101</pre> conecta al nodo equivocado al configurar nodo07	nodo01 sigue corriendo y ambos tienen la IP <code>.101</code> al mismo tiempo	Apagar nodo01 con <code>VBoxManage controlvm "bdd-nodo01" acpipowerbutton</code> antes de arrancar nodo07; esperar 20 segundos
<pre>VBoxManage clonevm falla con Could not find a snapshot named 'fase06- completa'</pre>	El nombre del snapshot difiere del esperado	Ejecutar <code>VBoxManage snapshot "bdd-nodo01" list</code> y copiar el nombre exacto; ajustar el comando <code>clonevm</code>
<pre>mariadb -h 192.168.56.106 -u app_final desde bdd- cliente produce ERROR 1045 (28000)</pre>	El usuario <code>app_final</code> fue creado solo para el host <code>.1</code> pero no para <code>.107</code>	En nodo06: ejecutar el bloque E.12 completo para crear <pre>app_final@'192 .168.56.107'</pre>

La Prueba 8b muestra <code>ERROR 2003 (HY000): Can't connect</code> en lugar de <code>ERROR 1045</code>	El puerto 3306 de nodo04 está bloqueado por firewall	Verificar con <code>sudo ufw status</code> en nodo04; si está activo, añadir <code>sudo ufw allow from 192.168.56.107 to any port 3306</code>; el resultado esperado debe ser <code>ERROR 1045</code>
<code>CALL analizar_consulta('norte', FALSE)</code> falla con <code>ERROR 1305: PROCEDURE does not exist</code>	El procedimiento no se creó correctamente	Ejecutar <code>SHOW PROCEDURE STATUS WHERE Db = 'lab_bdd'</code>; si no aparece, repetir el bloque E.8 completo incluyendo el <code>DELIMITER //</code>
<code>netplan apply</code> falla al configurar nodo07	Problemas de indentación YAML	Usar el comando <code>sed -i 's/...'</code> sin edición manual; validar con <code>sudo netplan try</code> antes de aplicar
Las pruebas de transparencia 3-7 devuelven 0 filas	Spider en nodo06 no alcanza los shards porque nodo04/nodo05 no están encendidos	Verificar con <code>VBoxManage list runningvms</code> que nodo04, nodo05 y nodo06 estén activos
El snapshot de nodo07 falla con	La VM está en ejecución al momento de	Apagar nodo07 con <code>sudo poweroff</code>, esperar que el

VBOX_E_INVALID
_VM_STATE

tomar el
snapshot

estado cambie a
powered off y
reintentar

H. Checklist de validación

- ☐ `SHOW TABLES IN lab_bdd` en nodo04 no incluye `productos`; muestra exactamente `clientes`, `detalle_pedidos`, `pedidos`, `v_productos_basico`.
- ☐ `SHOW TABLES IN lab_bdd` en nodo05 no incluye `productos`; muestra exactamente `clientes`, `detalle_pedidos`, `pedidos`, `v_productos_detalle`.
- ☐ `SELECT COUNT(*) FROM v_productos_basico` en nodo06 devuelve **10**.
- ☐ `SELECT COUNT(*) FROM v_productos_detalle` en nodo06 devuelve **10**.
- ☐ `SELECT COUNT(*) FROM productos` (VIEW) en nodo06 devuelve **10** tras el DROP en los shards.
- ☐ `ANALYZE TABLE` se ejecutó en todas las tablas de nodo04 y nodo05 sin errores.
- ☐ La tabla de costes generada en E.3 muestra valores de `bytes_totales_est > 0` para cada fragmento.
- ☐ La simulación de la Fase 4 (E.4) devuelve **4 filas** idénticas a las de la consulta H-6 directa.
- ☐ Las estrategias A y B de semijoin (E.5) devuelven el **mismo resultado de negocio**.
- ☐ `EXPLAIN SELECT * FROM clientes WHERE region='norte'` en nodo06 muestra **solo frag_A**.
- ☐ `EXPLAIN SELECT * FROM clientes` sin filtro en nodo06 muestra **frag_A y frag_B**.
- ☐ `EXPLAIN SELECT COUNT(*) FROM detalle_pedidos` en nodo06 no muestra poda de partición.
- ☐ `EXPLAIN SELECT id FROM detalle_pedidos WHERE pedido_id IN (1,2,3)` en nodo04 (directo) muestra uso del índice `idx_pedido`.
- ☐ La versión `SELECT *` produce un `Bytes_sent` mayor que la versión con proyección explícita.
- ☐ El procedimiento `analizar_consulta` existe en `information_schema.ROUTINES` con `ROUTINE_SCHEMA = 'lab_bdd'`.
- ☐ `CALL analizar_consulta('norte', FALSE)` devuelve `grado_de_localidad = 0`.
- ☐ `CALL analizar_consulta(NULL, TRUE)` devuelve `grado_de_localidad = 2`.
- ☐ `CALL analizar_consulta('sur', TRUE)` devuelve `grado_de_localidad = 1`.
- ☐ `VBoxManage showvminfo "bdd-nodo07"` confirma la VM registrada como clon enlazado.
- ☐ `hostname` en nodo07 devuelve `bdd-cliente`.

- ☐ `ip addr show | grep 192.168.56` en nodo07 muestra `192.168.56.107/24`.
- ☐ `systemctl status mariadb` en nodo07 indica que el servicio no existe (cliente solamente).
- ☐ `which mariadb` en nodo07 devuelve `/usr/bin/mariadb`.
- ☐ La Prueba 2 desde bdd-cliente lista las tablas y VIEWS del esquema `lab_bdd`.
- ☐ La Prueba 3 desde bdd-cliente devuelve **4 filas** con 5 clientes cada una.
- ☐ La Prueba 4 desde bdd-cliente devuelve **5 clientes** con `region = 'norte'`.
- ☐ La Prueba 5 desde bdd-cliente devuelve **10 productos** con columnas de basico y detalle combinadas.
- ☐ La Prueba 6 desde bdd-cliente devuelve **4 filas** con facturación por región.
- ☐ `CALL consulta_regional(NULL)` desde bdd-cliente devuelve **4 filas**.
- ☐ `CALL consulta_regional('este')` desde bdd-cliente devuelve **1 fila**.
- ☐ La Prueba 8a produce error de acceso denegado a `mysql.servers`.
- ☐ Las Pruebas 8b y 8c producen error de acceso denegado al intentar conectar a nodo04 y nodo05.
- ☐ Los snapshots `fase16-completa` existen en los **siete nodos** del laboratorio.
- ☐ Puedo enumerar de memoria las cuatro fases del procesamiento de consultas distribuidas.
- ☐ Puedo definir el semijoin distribuido y la condición que lo hace rentable (`size(S) > size( $\pi_a(S)$ ) + size( $R \times S$ )`).
- ☐ Puedo explicar por qué la VIEW `detalle_pedidos` (UNION ALL) no se beneficia del predicate pushdown automático de Spider.

Preguntas teóricas para estudiantes

1. En la Fase 3 (optimización global) del bloque E.4, se eligió la estrategia de sub-agregación por shard: cada shard devuelve solo cuatro filas (una por región de su fragmento) en lugar de todas las filas brutas de `clientes`, `pedidos` y `detalle_pedidos`. Analiza cuándo esta estrategia es beneficiosa y cuándo no: ¿qué ocurre si la consulta no tiene un `GROUP BY`?, ¿qué ocurre si el `GROUP BY` es sobre una columna que no es el atributo de fragmentación (por ejemplo, `GROUP BY pedidos.estado` en lugar de `GROUP BY clientes.region`)?, ¿puede el coordinador Spider aplicar esta sub-agregación automáticamente o siempre tiene que implementarla el desarrollador de forma manual?
2. El algoritmo de semijoin de Bernstein-Chiu resultó no ser rentable para el esquema actual del laboratorio porque todos los productos están referenciados en `detalle_pedidos`. Describe un escenario de negocio realista —utilizando exactamente el esquema `lab_bdd`

con algunas modificaciones de datos— en el que el semijoin sería altamente rentable: ¿cuántos productos habría que agregar al catálogo?, ¿cuántos de ellos tendrían pedidos activos?, ¿cuántas líneas de `detalle_pedidos` habría?, y ¿cuál sería el porcentaje exacto de reducción de datos transferidos entre el coordinador y nodo04?

3. La poda por índice en nodo04 (E.7) mostró que con fragmentos de 10 filas el optimizador InnoDB puede preferir un full scan sobre el uso del índice compuesto. Explica el modelo de coste interno que usa InnoDB para tomar esa decisión: ¿qué variable de configuración controla el umbral entre el acceso por índice y el full scan?, ¿en qué unidad se mide el coste (filas, bloques de disco, tiempo)?, ¿a partir de qué tamaño de fragmento esperaría que InnoDB eligiera el índice `idx_region_cliente (region, cliente_id)` de forma consistente? Apoya la respuesta con la salida de `EXPLAIN` que obtuviste en el laboratorio.
4. La transparencia de distribución verificada en las Pruebas 1–8 cubre tres de los cuatro tipos definidos en la Fase 0 (fragmentación, ubicación y replicación). El cuarto tipo —transparencia de concurrencia— no fue verificado en esta fase. Diseña un escenario concreto de prueba de transparencia de concurrencia que podría ejecutarse desde `bdd-cliente`: ¿qué dos operaciones simultáneas demostrarían que el sistema gestiona la concurrencia de forma transparente para el cliente?, ¿qué tablas y qué tipo de operaciones estarían involucradas?, ¿en qué fase del laboratorio se realizará esta prueba y por qué se dejó para ese momento?
5. El procedimiento `analizar_consulta()` calcula el coste de transferencia usando la fórmula `filas × bytes_por_fila`. En un sistema de producción real, el coste de comunicación en una red WAN incluye también la latencia de ida y vuelta (*round-trip time*, RTT). Dado que la fórmula de coste total sería $C = latencia \times num_mensajes + filas \times bytes_por_fila / ancho_de_banda$, analiza cómo cambia la estrategia óptima cuando la latencia domina sobre el volumen de datos: ¿en qué situaciones el semijoin deja de ser beneficioso aunque reduzca el volumen de datos?, ¿por qué en la red Host-Only del laboratorio este factor no es observable?, ¿qué parámetro podría configurarse en la red virtual de VirtualBox para simular latencia artificialmente?

Ejercicios prácticos

1. **Implementación completa del algoritmo de descomposición para una consulta nueva.** Seleccionar la consulta H-7 del catálogo de la Fase 15 (reporte analítico avanzado con peso y descripción de producto) y aplicar manualmente las cuatro fases de descomposición: (a) escribir la expresión de álgebra relacional con proyecciones tempranas empujadas hacia las hojas, (b) sustituir cada relación global por su expresión de fragmentos con los predicados simplificados, © describir en texto el plan de optimización

global que minimiza el volumen de datos transferidos, y (d) implementar la Fase 4 (ejecución distribuida manual) como consultas SQL en nodo06 usando las tablas Spider directas en lugar de las VIEWS, verificando que el resultado sea idéntico a H-7. Documentar el coste estimado en KB para cada etapa.

2. **Benchmark de semijoin en un catálogo extendido.** Insertar 40 productos adicionales en `v_productos_basico` de nodo04 (con IDs del 11 al 50) sin insertar líneas de detalle para ellos. El catálogo pasa a tener 50 productos de los cuales solo 10 tienen pedidos activos. Repetir el experimento de semijoin de E.5 con el catálogo extendido: (a) medir los `Handler_read_rnd_next` de la Estrategia A y la Estrategia B, (b) calcular el porcentaje de reducción real de filas accedidas en `v_productos_basico`, © verificar que la condición de rentabilidad `size(S) > size( $\pi_a(S)$ ) + size( $R \bowtie S$ )` ahora se cumple con los nuevos valores, y (d) eliminar los 40 productos de prueba al finalizar. Documentar todos los pasos con las salidas de `SHOW STATUS`.
3. **Simulación de sesión completa de aplicación desde bdd-cliente.** Escribir un script Bash en `bdd-nodo07` (`/home/bddadmin/app_simulacion.sh`) que simule una sesión completa de la aplicación de negocio: (a) conectar a nodo06 como `app_final`, (b) consultar el número de clientes por región, © buscar pedidos entregados del mes más reciente, (d) obtener la ficha completa de los tres productos más vendidos usando la VIEW `productos`, (e) calcular el ticket promedio por región vía `CALL consulta_regional(NULL)`, y (f) imprimir un reporte de resumen formateado en texto plano. El script no debe incluir en ningún lugar las IPs de nodo04 ni nodo05 ni referencias a la fragmentación interna. Ejecutarlo y documentar la salida completa.

Reto adicional para alumnos avanzados

Implementar en `bdd-nodo06` un **motor de enrutamiento automático de semijoin** mediante una tabla de metadatos y un procedimiento genérico:

Crear la tabla `lab_bdd.plan_semijoin` con las columnas `tabla_grande`, `tabla_pequeña`, `atributo_join`, `selectividad_est` (fracción de filas de `tabla_grande` que coinciden con `tabla_pequeña`) y `aplicar_semijoin` (BOOLEAN calculado automáticamente cuando la condición de rentabilidad se cumple).

Crear el procedimiento `evaluar_semijoin(p_tabla_grande, p_tabla_pequeña, p_atributo)` que: (1) consulta `information_schema.TABLES` para obtener `TABLE_ROWS` y `AVG_ROW_LENGTH` de ambas tablas, (2) estima `size( $\pi_a(S)$ )` asumiendo 4 bytes por ID, (3) estima `size( $R \bowtie S$ )` usando la selectividad almacenada en `plan_semijoin`, (4) evalúa la condición de

rentabilidad `size(S) > size( $\pi_a(S)$ ) + size(R  $\times$  S)`, y (5) actualiza `plan_semijoin.aplicar_semijoin` y devuelve un resultado con la recomendación y los valores intermedios.

Poblar `plan_semijoin` con las cuatro combinaciones relevantes del esquema (`detalle_pedidos  $\times$  v_productos_basico`, `pedidos  $\times$  clientes`, etc.), ejecutar `CALL evaluar_semijoin(...)` para cada una y documentar cuáles resultan rentables con el dataset actual de 10–36 filas por fragmento y cuáles lo serían si los fragmentos tuvieran 10 000 filas con la misma selectividad.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Eliminación de redundancia (<code>DROP TABLE productos</code>)	5%	La tabla <code>productos</code> original no existe en nodo04 ni nodo05; las VIEWS y tablas Spider del coordinador siguen funcionando sin interrupción tras el DROP
Análisis estadístico de costes	10%	<code>ANALYZE TABLE</code> ejecutado en todos los fragmentos; el estudiante construye y puede interpretar la tabla de costes consolidada, relacionando <code>TABLE_ROWS</code> × <code>AVG_ROW_LENGTH</code> con el volumen de transferencia de red
Simulación de las cuatro fases de descomposición	25%	Las cuatro fases se ejecutan en SQL correctamente; el resultado de la Fase 4 manual es idéntico al de H-6 directa; el estudiante

		puede explicar qué transformación algebraica ocurre en cada fase y cuál es el aporte de la sub-agregación por shard
Implementación de semijoin distribuida	20%	Las Estrategias A y B producen el mismo resultado; el estudiante mide y compara los <code>Handler_read_*</code>; puede evaluar la condición de rentabilidad del semijoin con los valores reales del dataset
Análisis EXPLAIN y métricas de ejecución	10%	Los EXPLAIN demuestran poda en tablas particionadas y ausencia de poda en la VIEW UNION ALL; el estudiante compara <code>Bytes_sent</code> entre <code>SELECT *</code> y proyección explícita
Procedimiento <code>analizar_consulta()</code>	10%	El procedimiento existe, produce resultados correctos para los cuatro casos de prueba

Provisionamiento y configuración de <code>bdd-cliente</code>	10%	nodo07 existe con IP <code>.107</code> , hostname <code>bdd-cliente</code> , sin servidor MariaDB, con <code>mariadb-client</code> instalado; conecta correctamente a nodo06
Pruebas de transparencia desde <code>bdd-cliente</code>	10%	Las ocho pruebas producen los resultados esperados; las pruebas 8a-8c confirman que el cliente no puede acceder a la infraestructura interna

I. Preparación para la siguiente fase

La **Fase 17: Simulación de Fallos** requerirá:

- Los snapshots `fase16-completa` en los **siete nodos** del laboratorio (completados en esta fase).
- `bdd-nodo07` (`bdd-cliente`) operativo con `mariadb-client` instalado y conectividad verificada a nodo06: será el punto de observación externo desde donde se verificará el comportamiento del sistema ante fallos.
- Comprensión del estado de replicación del clúster Galera (nodo01–03): es el mecanismo de alta disponibilidad que la Fase 17 pondrá a prueba deliberadamente. Antes de iniciar la Fase 17 se deberá verificar que el clúster sigue en estado `Primary` con los tres nodos `Synced`.
- La arquitectura de siete nodos completa documentada en esta fase es el **escenario de referencia** para los experimentos de fallos: cada tipo de fallo (caída de shard, caída del coordinador, partición de red entre shards) produce un comportamiento distinto que la

Fase 17 analizará en términos del teorema CAP.

En la Fase 17 se simularán cuatro escenarios de fallo: (1) caída de un nodo del clúster Galera con verificación de quórum y recuperación automática, (2) caída de un shard (nodo04 o nodo05) y observación del comportamiento del coordinador Spider al intentar acceder al fragmento inaccesible, (3) caída del coordinador (nodo06) y recuperación con el estado de los shards intacto, y (4) desconexión de `bdd-cliente` durante una transacción en curso. Para cada escenario se documentará qué tipo de error recibe el cliente final, qué datos (si alguno) se pierden o quedan inaccesibles, y cuál es el procedimiento de recuperación.
